

Operaciones con Conjuntos en Python

Teoría y Implementación

Equipo: Conjuntos

9 de octubre de 2025

Contenido

- 1 Introducción
- 2 Operaciones Básicas
- 3 Operaciones de Conjunto
- 4 Operaciones de Comparación
- 5 Métodos de Actualización
- 6 Aplicaciones Prácticas
- 7 Resumen y Mejores Prácticas

¿Qué son los Conjuntos en Python?

Definición

Estructura de datos que almacena elementos **únicos** y **no ordenados**

- Similar a los conjuntos matemáticos
- No permiten elementos duplicados
- No mantienen un orden específico
- Optimizados para operaciones de pertenencia

Creación de conjuntos

```
Directamente conjunto2 = 1, 2, 3, 4, 5  
Conjunto vaco vacio = set()
```

add() - Agregar Elementos

Función

Agrega un elemento al conjunto

Ejemplos

```
numeros = {1, 2, 3}

# Agregar un elemento
numeros.add(4)
print(numeros)  # {1, 2, 3, 4}

# Intentar agregar duplicado
numeros.add(2)
print(numeros)  # {1, 2, 3, 4} (sin cambios)

# Agregar mltiples elementos
numeros.add(5)
numeros.add(6)
print(numeros)  # {1, 2, 3, 4, 5, 6}
```

remove() vs discard()

Diferencias

- `remove(x)`: Elimina `x`, genera error si no existe
- `discard(x)`: Elimina `x`, no genera error si no existe

Ejemplos Comparativos

```
conjunto = {1, 2, 3, 4, 5}

# remove() - genera error si el elemento no existe
conjunto.remove(3)
print(conjunto)    # {1, 2, 4, 5}

# discard() - no genera error
conjunto.discard(2)
print(conjunto)    # {1, 4, 5}

conjunto.discard(10)    # No pasa nada
# conjunto.remove(10)    # KeyError!
```

clear() y pop()

clear()

Elimina todos los elementos

```
conjunto = {1, 2, 3}
print(conjunto) # {1, 2, 3}

conjunto.clear()
print(conjunto) # set()
```

pop()

Elimina y retorna un elemento aleatorio

```
conjunto = {1, 2, 3, 4, 5}

while conjunto:
    elemento = conjunto.pop()
    print(f"Removido: {elemento}")

print(conjunto) # set()
```

copy() - Copia de Conjuntos

¡Importante!

La asignación directa crea una referencia, no una copia

Diferencia entre copy() y asignación

```
original = {1, 2, 3, 4, 5}

# copy() crea un nuevo conjunto
copia = original.copy()

# Asignación crea una referencia
referencia = original

# Modificamos el original
original.add(6)

print("Original:", original)
print("Copia:", copia)           # No cambia
print("Referencia:", referencia) # Cambia
```

union() - Unión de Conjuntos

Definición Matemática

$$A \cup B = \{x \mid x \in A \vee x \in B\}$$

Ejemplos Python

```
A = {1, 2, 3}
B = {3, 4, 5}

# M todo union()
union1 = A.union(B)
print(union1)  # {1, 2, 3, 4, 5}

# Operador |
union2 = A | B
print(union2)  # {1, 2, 3, 4, 5}

# M ltiples conjuntos
C = {5, 6, 7}
union3 = A.union(B, C)
print(union3)  # {1, 2, 3, 4, 5, 6, 7}
```


intersection() - Intersección

Definición Matemática

$$A \cap B = \{x \mid x \in A \wedge x \in B\}$$

Ejemplos Python

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

# M todo intersection()
interseccion1 = A.intersection(B)
print(interseccion1)  # {3, 4}

# Operador &
interseccion2 = A & B
print(interseccion2)  # {3, 4}

# M multiples conjuntos
C = {4, 5, 7}
interseccion3 = A.intersection(B, C)
print(interseccion3)  # {4}
```

difference() - Diferencia

Definición Matemática

$$A - B = \{x \mid x \in A \wedge x \notin B\}$$

Ejemplos Python

```
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7}

# Diferencia A - B
diff1 = A.difference(B)
print(diff1) # {1, 2, 3}

# Operador -
diff2 = A - B
print(diff2) # {1, 2, 3}

# Diferencia B - A
diff3 = B - A
print(diff3) # {6, 7}
```

symmetric_difference()

Definición Matemática

$$A \Delta B = (A - B) \cup (B - A)$$

Ejemplos Python

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

# M todo symmetric_difference()
sym_diff1 = A.symmetric_difference(B)
print(sym_diff1)  # {1, 2, 5, 6}

# Operador ^
sym_diff2 = A ^ B
print(sym_diff2)  # {1, 2, 5, 6}

# Propiedad conmutativa
sym_diff3 = B ^ A
print(sym_diff3)  # {1, 2, 5, 6}
```

issubset() - Subconjunto

Definición Matemática

$$A \subseteq B \iff \forall x (x \in A \implies x \in B)$$

Ejemplos Python

```
A = {1, 2}
B = {1, 2, 3, 4}
C = {1, 5}

print(A.issubset(B))    # True
print(A <= B)           # True
print(C.issubset(B))    # False

# Subconjunto propio
print(A < B)             # True
print(A < A)             # False
print(A <= A)            # True
```

issuperset() - Superconjunto

Definición Matemática

$$A \supseteq B \iff B \subseteq A$$

Ejemplos Python

```
A = {1, 2, 3, 4}
B = {1, 2}
C = {1, 5}

print(A.issuperset(B))    # True
print(A >= B)             # True
print(A.issuperset(C))    # False

# Superconjunto propio
print(A > B)              # True
print(A > A)              # False
print(A >= A)             # True
```

isdisjoint() - Conjuntos Disjuntos

Definición Matemática

$$A \cap B = \emptyset$$

Ejemplos Python

```
A = {1, 2, 3}
B = {4, 5, 6}
C = {3, 4, 5}

print(A.isdisjoint(B))    # True
print(A.isdisjoint(C))    # False
print(B.isdisjoint(C))    # False

# Con conjunto vacio
vacio = set()
print(A.isdisjoint(vacio)) # True
```

update() y Variantes

Característica

Modifican el conjunto original (in-place)

Ejemplos

```
A = {1, 2, 3}
B = {3, 4, 5}

# update() - union in-place
A.update(B)
print(A)  # {1, 2, 3, 4, 5}

# intersection_update()
A = {1, 2, 3, 4}
A.intersection_update(B)
print(A)  # {3, 4}

# difference_update()
A = {1, 2, 3, 4, 5}
A.difference_update(B)
print(A)  # {1, 2}
```

Análisis de Usuarios

```
# Usuarios en diferentes plataformas
facebook = {"Ana", "Carlos", "Diana", "Eva"}
twitter = {"Carlos", "Diana", "Fran", "Gabo"}
instagram = {"Ana", "Eva", "Gabo", "Hector"}

# Usuarios en todas las plataformas
todas = facebook & twitter & instagram
print("En todas:", todas)  # set()

# Usuarios nicos de Instagram
solo_instagram = instagram - facebook - twitter
print("Solo Instagram:", solo_instagram)  # {'Hector'}

# Total de usuarios nicos
total_usuarios = facebook | twitter | instagram
print("Total usuarios:", len(total_usuarios))  # 7
```


Ejemplo: Eliminación de Duplicados

Limpieza de Datos

```
# Lista con duplicados
emails = [
    "user@example.com", "admin@test.com",
    "user@example.com", "info@demo.com",
    "admin@test.com", "support@site.com"
]

print("Original:", len(emails), "emails")  # 6 emails

# Eliminar duplicados usando conjuntos
emails_unicos = list(set(emails))
print("Unicos :", len(emails_unicos), "emails")  # 4 emails

# Mantener orden (Python 3.7+)
from collections import OrderedDict
emails_ordenados = list(OrderedDict.fromkeys(emails))
print("Ordenados:", emails_ordenados)
```

Ejemplo: Sistema de Recomendación

Intereses de Usuarios

```
# Intereses de usuarios
usuario1 = {"python", "ml", "data science", "ai"}
usuario2 = {"python", "web development", "javascript"}
usuario3 = {"java", "mobile", "android"}

# Encontrar intereses comunes
comunes_1_2 = usuario1 & usuario2
print("Comunes usuario1-2:", comunes_1_2)  # {'python'}

# Recomendaciones basadas en intereses
recomendaciones_1 = usuario2 - usuario1
print("Recomendaciones para usuario1:", recomendaciones_1)
# {'web development', 'javascript'}

# Usuarios con intereses similares
similares = usuario1 | usuario2
print("Intereses similares:", similares)
```

Tabla Resumen de Operaciones

Operación	Símbolo	Método	Descripción
Unión		<code>union()</code>	Elementos en A o B
Intersección	&	<code>intersection()</code>	Elementos en A y B
Diferencia	—	<code>difference()</code>	Elementos en A pero no en B
Dif. Simétrica	$\wedge$	<code>symmetric_difference()</code>	Elementos en A o B pero no en ambos
Subconjunto	$\leq$	<code>issubset()</code>	Todos los elementos de A están en B
Superconjunto	$\geq$	<code>issuperset()</code>	B contiene todos los elementos de A
Disjuntos	-	<code>isdisjoint()</code>	No comparten elementos

Consejos de Uso

- **Operadores vs Métodos:** Usa operadores para código más limpio
- **Copia vs Referencia:** Siempre usa `copy()` para duplicados
- **Eliminación Segura:** Prefiere `discard()` sobre `remove()`
- **Eliminación de Duplicados:** Conjuntos son la forma más eficiente
- **Actualizaciones In-place:** Usa métodos `update()` para mejor performance

Ventajas de Rendimiento

- Búsqueda de elementos: $O(1)$
- Inserción: $O(1)$
- Eliminación: $O(1)$
- Operaciones de conjunto: $O(n)$

¿Cuándo usar conjuntos?

- Cuando necesitas elementos únicos
- Para pruebas rápidas de pertenencia
- Para eliminar duplicados
- Para operaciones matemáticas entre colecciones

¿Cuándo NO usar conjuntos?

- Cuando necesitas orden
- Cuando necesitas elementos indexados
- Cuando trabajas con datos duplicados
- Cuando los elementos no son hasheables

¡Gracias!

¿Preguntas?